arXiv:2603.18464v1 [cs.LG] 19 Mar 2026

AcceRL: A Distributed Asynchronous Reinforcement Learning and World Model Framework for Vision-Language-Action Models

Chengxuan Lu[1,*] **Shukuan Wang**[1,*] **Yanjie Li**[1] **Wei Liu**[1] **Shiji Jin**[1] **Fuyuan Qian**[1]
Peiming Li[1] **Baigui Sun**[1,†] **Yang Liu**[1,†]

Abstract

Reinforcement learning (RL) for large-scale Vision-Language-Action (VLA) models faces significant challenges in computational efficiency and data acquisition. We propose AcceRL, a fully asynchronous and decoupled RL framework designed to eliminate synchronization barriers by physically isolating training, inference, and rollouts. Crucially, AcceRL is the first to integrate a plug-and-play, trainable world model into a distributed asynchronous RL pipeline to generate virtual experiences. Experiments on the LIBERO [1] benchmark demonstrate that AcceRL achieves state-of-the-art (SOTA) performance. Systematically, it exhibits super-linear scaling in throughput and highly efficient hardware utilization. Algorithmically, the world-model-augmented variant delivers unprecedented sample efficiency and robust training stability in complex control tasks.

Keywords Asynchronous Reinforcement Learning · Embodied Intelligence · World Model

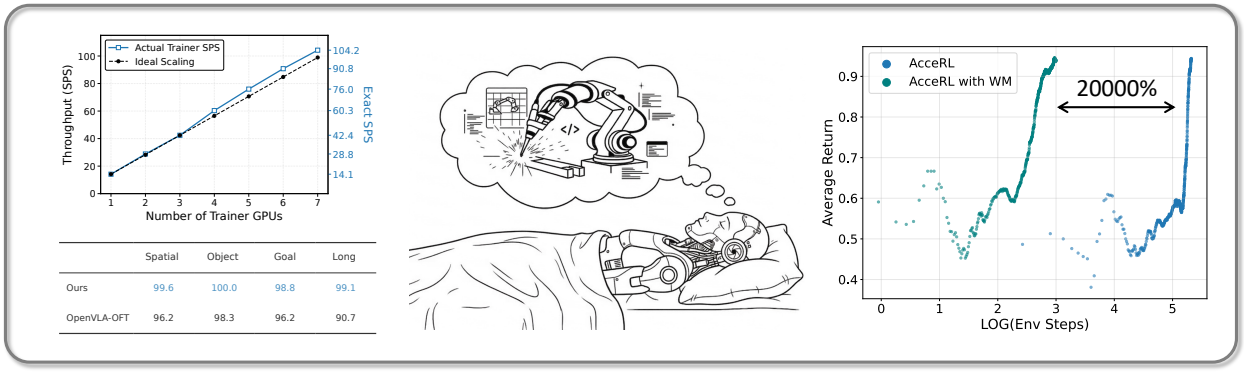


Figure 1: **Overview of AcceRL**. (Top Left) The framework's throughput exhibits super-linear scaling with the number of trainer GPUs, enabled by ZeRO [2] optimizations. (Bottom Left) Experimental results on the LIBERO benchmark demonstrate that AcceRL achieves SOTA performance across all evaluation categories. (Middle) A conceptual illustration of the world model in reinforcement learning, representing the agent "learning in imagination" to significantly boost data utilization. (Right) Performance comparison between model-based (AcceRL with WM) and model-free (AcceRL) approaches. By leveraging a world model pre-trained on 1,000 offline trajectories, the model-based architecture improves online sample efficiency by $200\times$.

[1]Wolf 1069B, Sany Group.
[*]Equal contribution.
[†]Corresponding authors: Baigui Sun (sunbaigui85@gmail.com) and Yang Liu (yang.15.liu@kcl.ac.uk).

1 Introduction

The field of embodied artificial intelligence has been profoundly transformed by the advent of Vision-Language-Action (VLA) models. These models promise to create agents capable of executing complex tasks based on natural language instructions. While imitation learning has enabled significant capabilities, it suffers from poor generalization and compounding errors known as covariate shift. To overcome these barriers, the research community has turned to reinforcement learning (RL) to optimize policies for long-term success and online fine-tuning.

However, applying RL to large-scale VLAs introduces severe systemic bottlenecks primarily due to the synchronization barriers in traditional frameworks. In a conventional synchronous setup, the system must frequently switch between environment interaction, policy inference, and model updates. This lockstep execution forces high-performance GPUs to idle while waiting for slow physical simulators. This creates a significant long-tail problem where the overall throughput is capped by the slowest rollout worker. Furthermore, even with optimized asynchronous execution, the training process remains constrained by the inherent low frequency of physical simulators and the extreme data hunger of large-scale VLA architectures. Relying solely on real-world or simulated interactions imposes a prohibitive temporal cost because the sampling speed is strictly limited by the simulator stepping and the model inference latency. There is an urgent need for a mechanism that can break free from the temporal constraints of physical simulation by generating high-fidelity synthetic experiences to accelerate policy refinement.

To bridge this gap, we introduce AcceRL, a modular and fully asynchronous framework tailored for large-scale VLA models. AcceRL achieves a paradigm shift by physically isolating training, inference, and sampling into independent asynchronous streams. Our key contributions are as follows:

- High-throughput asynchronous RL architecture: We propose a fully decoupled infrastructure that removes synchronization barriers. By eliminating frequent mode-switching and mitigating long-tail latencies, AcceRL maximizes hardware utilization and achieves super-linear scaling in system throughput.
- First distributed asynchronous framework with a trainable world model: AcceRL seamlessly incorporates both world model training and plug-and-play inference pools directly into its pipeline. By enabling "learning in imagination," the framework bypasses high-cost real-world interactions and continuously refines the world model alongside the policy. This architectural innovation improves online sample efficiency by an astonishing $200\times$ (20,000%).
- Empirical excellence and SOTA performance: Evaluated on the challenging LIBERO benchmark, AcceRL achieves state-of-the-art (SOTA) task performance. The framework demonstrates both highly efficient systemic scalability and robust training stability in complex embodied control tasks.

2 Related work

2.1 Evolution of Distributed Reinforcement Learning Frameworks

Historically, scaling reinforcement learning has relied on distributed frameworks to accelerate data acquisition. Even contemporary large-scale systems, such as the default veRL framework [3] and the synchronous baselines evaluated in RLinf [4] [5], still rely on a synchronous mechanism. In this lockstep paradigm, the central learner must wait for all distributed rollout workers to finish collecting trajectories before performing a gradient update. While this strict synchronization prevents severe trajectory staleness and maintains the bounded policy lag required by algorithms like PPO [6], the overall system throughput is dictated by the slowest worker. This phenomenon is known as the straggler effect [7], which is particularly exacerbated in large-scale generative tasks due to highly variable trajectory lengths. As a result, expensive compute resources are frequently left idle.

To mitigate these synchronization overheads, frameworks evolved towards asynchronous architectures. General-purpose systems like Ray RLlib [8] introduced asynchronous task scheduling. However, their underlying architectures were fundamentally designed for lightweight models. When applied to multi-billion parameter Vision-Language-Action (VLA) models, they lack native deep integration with large-model optimization techniques such as efficient parameter offloading or model parallelism, leading to prohibitive memory bottlenecks. Addressing this coupling, IMPALA [9] pioneered actor-learner decoupling; yet, by retaining model inference on rollout workers, it still incurs massive memory costs when deploying such large models. Later, Laminar [10] introduced a fully decoupled architecture with trajectory-level asynchrony to solve long-tail latency. However, its design is tailored for closed-loop autoregressive generation within memory, making it structurally misaligned with the external, step-by-step causal dependencies of physical simulators required by embodied agents. Most recently, RL-VLA^3 [11] adapted these principles to Embodied AI by separating environment simulation from inference to saturate GPU utilization. While optimizing execution, it remains fundamentally bottlenecked by the slow speed of the physical simulator itself. This historical evolution highlights a

critical unmet need for a fully decoupled architecture that can bypass the inherent latency of physical simulators to scale embodied foundation models.

However, their design is primarily tailored for closed-loop autoregressive generation entirely within memory, making them structurally misaligned with the external, step-by-step causal dependencies required by embodied agents interacting with physical simulators. To bridge this gap, domain-specific frameworks have emerged. For instance, RLinf-VLA [5] introduces a hybrid fine-grained pipeline to overlap rendering and training for VLA models. Yet, it remains fundamentally bottlenecked by the inherently slow stepping speed of the external physical simulator itself. Similarly, RL-VLA^3 [11] attempts to saturate GPU utilization by strictly decoupling environment simulation from model inference via asynchronous queues. While optimizing raw execution speed, this asynchronous architecture exposes two critical vulnerabilities: it inevitably introduces severe trajectory staleness and massive cross-node communication overhead for high-dimensional multimodal data, thereby significantly degrading sample efficiency.

In contrast, AcceRL extends this architectural decoupling to a fundamentally new paradigm by integrating a plug-and-play World Model. This allows the system to bypass the high-latency physical simulators through zero-offload, distributed imagination rollouts. By permanently keeping both the policy and world models on dedicated GPU inference pools, AcceRL eliminates the severe VRAM swapping bottlenecks of collocated execution while resolving the external I/O dependency. This evolution—from strict synchronous locking, to compromised partial decoupling, and finally to AcceRL's unified separation of training, inference, and scalable world-model simulation—highlights a robust pathway for scaling embodied foundation models without sacrificing algorithmic fidelity.

2.2 Training and Optimization of Vision-Language-Action Models

The remarkable reasoning capabilities of Large Language Models (LLMs) have recently been augmented with visual encoders, giving rise to Vision-Language Models (VLMs). Integrating these multimodal foundation models into robotics has created a new paradigm where physical control is treated as a sequence modeling task. Pioneering frameworks like RT-1 [12] and RT-2 [13] demonstrate that Transformer architectures can control robots by discretizing actions into text-like tokens. These models are primarily trained using Behavior Cloning (BC) on large offline datasets. However, BC suffers from a fundamental limitation known as covariate shift [14]; small errors during execution lead the robot to unfamiliar states, causing compounding failures that static training data cannot correct.

To address these errors, Reinforcement Learning (RL) is often required for online fine-tuning. However, applying RL to VLA models introduces severe efficiency bottlenecks. For instance, recent works like SimpleVLA-RL [15] have attempted to adapt LLM-based RL frameworks to VLA models, demonstrating performance gains using outcome-based rewards. However, SimpleVLA-RL stands as a typical representative of the synchronous training scheme. Its fundamental reliance on a strict collect-then-update mechanism dictates that the central learner must consistently halt and wait for all distributed rollout workers to finish their environment interactions before parameter updating. Due to the inherent low frequency of physical simulators coupled with the heavy inference latency of multi-billion parameter VLAs, this synchronous barrier causes severe GPU idling and straggler effects. Consequently, the training process becomes prohibitively slow and computationally inefficient. To lower the hardware barrier, recent open-source efforts like OpenVLA [16] and Octo [17] often incorporate parameter-efficient fine-tuning (PEFT) techniques, such as Low-Rank Adaptation (LoRA) and quantization. While these methods lower the hardware barrier for training, they do not solve the inference latency problem. RL fine-tuning necessitates generating millions of rollout steps. Furthermore, as modern VLAs increasingly adopt multi-step diffusion or flow-based action heads, applying off-policy RL introduces severe algorithmic instabilities, such as exploding gradients. While recent breakthroughs like SAC Flow [18] elegantly mitigate these issues via velocity-reparameterized sequential modeling, the underlying system-level bottleneck remains unaddressed: the sheer latency of generating rollouts with multi-step generative policies renders synchronous training pipelines severely underutilized. For a massive 7B-parameter model, running this many forward passes on a physical simulator is computationally expensive and prohibitively slow.

This highlights the dual need for better sample efficiency and a decoupled system architecture. World Models [19, 20] offer a solution by learning to predict the environment's dynamics. Instead of interacting with a slow physical simulator, the agent can learn inside a fast, latent dream space. Although generative environments like Genie [21] show great potential, integrating them into a distributed VLA training system remains a challenge. AcceRL bridges this gap by using a parallelized World Model to replace physical simulation, enabling efficient RL fine-tuning for large-scale embodied models.

2.3 World Models and Model-Based Reinforcement Learning

Model-Based Reinforcement Learning (MBRL) fundamentally accelerates convergence by learning a predictive model of the environment's dynamics, effectively substituting physical interactions with computational predictions. Early

approaches like the Dreamer series [20] projected observations into a compact latent space, using Recurrent Neural Networks (RNNs) to predict future states s_{t+1} and rewards r_{t+1}. By optimizing the policy within this differentiable dream, agents can experience millions of transition steps without a single real-world interaction, dramatically improving sample efficiency.

With the advent of generative AI, environmental prediction has evolved from simple latent dynamics to high-fidelity video generation powered by Diffusion Models [22]. Frameworks such as Genie [21] and Diamonds [23] leverage the iterative denoising process to model complex, multi-modal distributions of future frames. This capability is particularly critical for VLA models, as it allows the agent to visualize the consequences of fine-grained manipulation tasks that are difficult to capture with low-dimensional latent states. These diffusion-based world models act as an infinite, risk-free simulator, providing a dense stream of virtual samples that cover edge cases rarely encountered in the physical world.

However, integrating these powerful world models into the VLA training loop introduces major system-level bottlenecks due to VRAM constraints. Recent frameworks like WoVR [24] attempt to enable world-model-based RL for VLAs, but they rely on a collocated GPU allocation strategy. To avoid out-of-memory errors, these systems force synchronous phase transitions. This process is heavily bottlenecked by repeatedly moving model parameters between GPU and CPU memory (offloading and onloading). During these memory transfers, the GPU compute cores sit completely idle, causing severe hardware underutilization and low training throughput.

In contrast, AcceRL solves this problem by completely abandoning collocated execution. Instead, it uses a fully decoupled, physically isolated architecture that permanently keeps the policy and world models on dedicated GPUs. This "zero-offload" design reduces network communication to passing lightweight trajectory data. By doing so, AcceRL ensures that the GPU compute cores are continuously saturated, unlocking near-linear scalability for distributed rollout generation and remarkable super-linear training throughput for massive foundation models.

3 AcceRL: A Fully Asynchronous Training Framework

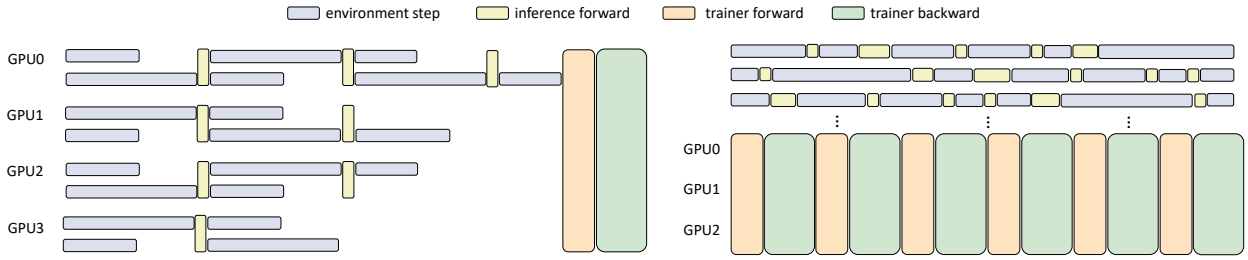


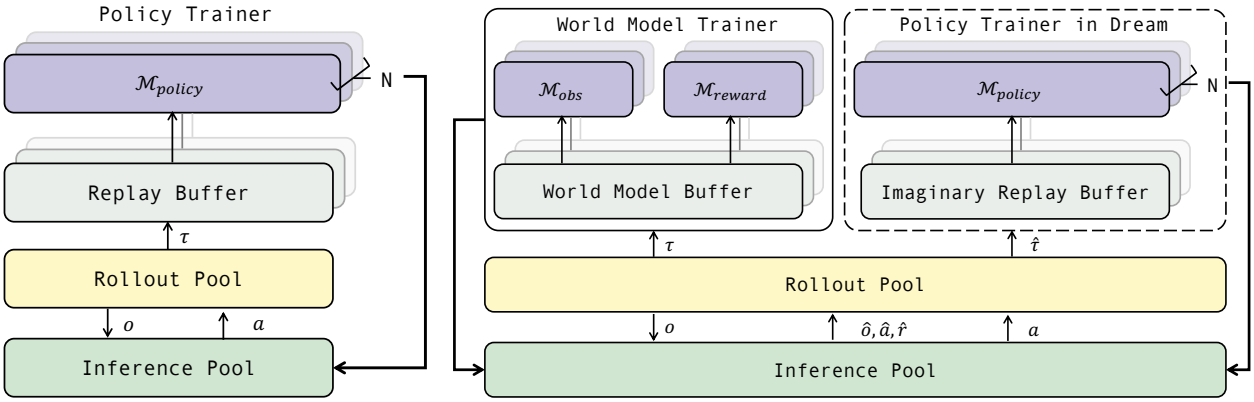
Figure 2: **Timeline of a synchronous RL framework (left) and the asynchronous framework AcceRL (right)**. The inference forward blocks in AcceRL encompass both the batching wait time and the inference time. By eliminating the GPU bubbles, AcceRL maximizes hardware utilization and significantly enhances overall training efficiency.

As shown in Figure 2, traditional synchronous RL frameworks suffer from heavy synchronization overhead caused by a cascading "long-tail" effect across three distinct levels:

- Step-Level Long-Tail: During environment interaction, the time required to compute a physical simulation step varies. In a lockstep setup, the GPU must wait for the slowest environment step to finish before executing a batched forward pass, creating severe micro-level idle bubbles.
- Episode-Level Long-Tail: Different environment instances terminate at different times due to early successes, failures, or varying task horizons. Synchronous frameworks force active environments to idle until all instances finish their current episodes, wasting significant compute cycles.
- Cluster-Level Long-Tail: At the macro scale, global barriers dictate that every rollout GPU must complete its entire data collection quota before the central model can perform a gradient update, meaning the entire cluster's throughput is capped by its slowest worker node.

As training scales to multi-billion parameter VLA models across large clusters, these three layers of latency compound exponentially, leading to prohibitive hardware underutilization.

To overcome these bottlenecks, we propose AcceRL (Figure 3a), a high-throughput asynchronous framework designed for large-scale Embodied AI that fundamentally redesigns the data flow through a dual-level asynchronous mechanism. This architecture achieves macro-asynchrony by decoupling the training process from rollout generation, while simultaneously ensuring micro-asynchrony by separating environment interaction from model inference.



(a) the base backbone

(b) the backbone augmented with world model

Figure 3: **The pipeline of AcceRL base backbone (left) and world-model-augmented mode (right).** The framework isolates training, inference, and rollout into independent asynchronous streams. By integrating a plug-and-play world model, AcceRL bypasses physical simulation constraints through imagination rollouts to significantly enhance sample efficiency.

3.1 Macro-Asynchrony: Decoupling Training and Rollout

To fundamentally resolve the cluster-level long-tail and eliminate global synchronization overhead, AcceRL introduces its first level of asynchrony: decoupling the Rollout Workers from the Trainer Workers. Unlike synchronous architectures that enforce a strict lockstep of "collect-then-update" across all nodes, AcceRL employs a non-blocking circular pipeline. Instead of waiting for all workers to finish their data collection quotas, Rollout Workers interact with environments independently and continuously stream completed trajectories to a distributed replay buffer $\mathcal{B}$. In parallel, Trainer Workers sample batches from $\mathcal{B}$ and update parameters continuously without ever blocking the data generation process. To further optimize large-scale training, the models within the Trainer Workers utilize ZeRO [2] to partition optimizer states and gradients across available GPUs, significantly improving memory efficiency. Upon the completion of each training batch, the updated parameters are synchronized to the corresponding Inference Actors via a high-performance NCCL broadcast group, ensuring that the rollout policy remains current without interrupting the training flow. By removing global synchronization barriers, GPUs minimize idle time waiting for data, allowing training to proceed at a steadier and more efficient pace.

3.2 Micro-Asynchrony: Decoupling Interaction and Inference

While macro-asynchrony removes global barriers, the second level of asynchrony addresses the inefficiency within the rollout process by eliminating the micro-level idle bubbles caused by step-level and episode-level long-tails. In traditional setups, a worker sequentially executes physical simulation steps and model inference. Because the computation time for distinct environment steps and the total horizon of different episodes vary significantly, this tightly coupled execution leads to frequent context switching, severe straggler effects, and poor GPU batching. To overcome this inefficiency, AcceRL adopts an Inference-as-a-Service paradigm to completely decouple physical interaction from policy inference. Each Rollout Worker manages an environment instance. When an observation o_t is generated, instead of computing the action locally, the worker sends an asynchronous request to a centralized Inference Pool and immediately suspends or handles other tasks. The Inference Workers maintain a request queue Q. To maximize GPU utilization while maintaining low latency, a batch forward pass is triggered based on a dynamic window mechanism. The trigger condition is defined as:

$$\text{Trigger} = (|Q| \geq B) \vee (t_{now} - t_{first} \geq T_{max}) \tag{1}$$

where B is the batch size and T_{max} is the maximum wait time. Once computed, actions are returned to the corresponding Rollout Workers to continue the rollout. After a sequence is completed, the collected data is packaged into a trajectory τ and stored in the replay buffer $\mathcal{B}$:

$$\tau = (o_{1:T+1}, a_{1:T}, r_{1:T}, \mu_{1:T}, v_{1:T}, \tilde{v}_{T+1}, \text{done}), \tag{2}$$

where the standardized notation is defined as follows:

- o_t: Observations (e.g., pixel-level images) received from the environment at time step t.

- a_t: Actions inferred by Inference Workers and executed by the Rollout Workers at time step t.
- r_t: Rewards received from the environment based on (o_t, a_t).
- μ_t: Policy logits used for off-policy correction.
- v_t: Value estimates for each observation o_t in the trajectory.
- $\tilde{v}_{T+1}$: Bootstrapped value for the observation at $T+1$, used for value targets.
- done: Termination flag indicating if the episode reached a natural end.

By decoupling environment interaction from model inference, the system enables the deployment of interaction tasks on cost-effective CPU platforms while centralizing inference on high-performance GPUs. This architecture ensures robust resource isolation and maximizes batching efficiency during the inference process.

3.3 Addressing Policy Lag

In an asynchronous distributed architecture, a significant challenge is policy lag, where the behavior policy μ used during rollout becomes stale compared to the current learner policy π. To mitigate the resulting off-policy bias and stabilize convergence, the TrainerWorker implements Value Re-computation, Global Advantage Normalization, and Gradient Calibration.

Value Re-computation For each trajectory τ retrieved from the replay buffer $\mathcal{B}$, the stale advantage estimates and value targets are discarded. Instead, the trainer re-evaluates the state values $V_\theta(o_t)$ using the most recent parameters θ:

$$V_t^{new} = V_\theta(o_t), \quad \forall t \in [0, T] \tag{3}$$

Based on these fresh value estimates, the Generalized Advantage Estimation (GAE) and the value targets are re-derived to align the advantage signal with the current value landscape:

$$A_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l (r_{t+l} + \gamma V_{t+l+1}^{new} - V_{t+l}^{new}) \tag{4}$$

This re-computation keeps the value baseline consistent with the current policy parameters before gradient computation. We provide a ablation study validating the critical importance of this step in Section 6.5.

Global Advantage Normalization To ensure training stability across the distributed training architecture, AcceRL implements a global normalization process after the advantage re-computation. Since sampling noise and task difficulties vary across different nodes, utilizing global statistics is essential to eliminate local biases and provide a unified baseline for policy updates.The process begins by aggregating local statistics from all nodes via a distributed AllReduce operation, including the sum of advantages (S_i), the squared sum (Q_i), and the sample count (N_i) for K nodes:

$$S_{total} = \sum_{i=1}^{K} S_k, \quad Q_{total} = \sum_{i=1}^{K} Q_k, \quad N_{total} = \sum_{i=1}^{K} N_k \tag{5}$$

Subsequently, the global mean μ_{global} and global variance σ^2_{global} are derived:

$$\mu_{global} = \frac{S_{total}}{N_{total}}, \quad \sigma^2_{global} = \frac{Q_{total}}{N_{total}} - \mu^2_{global} \tag{6}$$

Finally, the standardized advantages A_t are used for policy gradient computation to ensure stable optimization signals, thereby enhancing convergence efficiency and robustness in high-dimensional manipulation tasks:

$$A_t \leftarrow \frac{A_t - \mu_{global}}{\sqrt{\sigma^2_{global}} + \epsilon} \tag{7}$$

Gradient Calibration To further protect the policy from divergent updates when the lag is excessive, we employ Gaussian Importance sampling Policy Optimization (GIPO) [25]. Unlike standard PPO, which relies on hard clipping, GIPO introduces a Gaussian trust weight to softly damp extreme importance ratios. Given the ratio $\rho_t(\theta) = \frac{\pi(a_t|o_t)}{\mu(a_t|o_t)}$ and $\bar{\rho}_t = \text{sg}(\rho_t(\theta))$, the GIPO objective is defined as:

$$\omega(\bar{\rho}_t; \sigma) = \exp\left(-\frac{1}{2}\left(\frac{\log(\bar{\rho}_t)}{\sigma}\right)^2\right) \tag{8}$$

$$\mathcal{L}_\pi^{\text{GIPO}}(\theta) = -\mathbb{E}_{(o_t,a_t)\sim\mathcal{B}}\Big[\omega(\bar{\rho}_t; \sigma) \cdot \rho_t(\theta) \cdot A_t\Big] \tag{9}$$

where σ serves as a tunable hyperparameter controlling the width of the trust region. The total objective function is:

$$\min_{\theta,\phi} \mathcal{L}(\theta,\phi) = \mathcal{L}_{\pi}^{\text{GIPO}}(\theta) + \lambda_v \mathcal{L}_v(\phi) - \lambda_h \mathcal{L}_{\text{ent}}(\theta) \tag{10}$$

where $\mathcal{L}_v$ is the squared error loss for the value function and $\mathcal{L}_{\text{ent}}$ is an entropy bonus to encourage exploration. We provide a ablation study validating the empirical advantages of this algorithmic choice in Section 6.6.

3.4 Dynamic Weighted Resampling

To improve sample efficiency and balance the learning progress across diverse tasks, we implement a Dynamic Weighted Resampling (DWR) mechanism. DWR adaptively adjusts the sampling probability of each task based on its recent performance. For each task $i \in \{1, \ldots, N\}$, we maintain a sliding window of size H that records the outcomes of the most recent episodes. Let $s_{i,t} \in \{0, 1\}$ be the success indicator (1 for success, 0 for failure) for task i at step t. The sampling weight w_i for task i is formulated as the sum of failures within the window plus a Laplace smoothing factor:

$$w_i = \sum_{t=k-H+1}^{k} (1 - s_{i,t}) + \epsilon \tag{11}$$

The probability of selecting task i is then normalized as $P_i = w_i / \sum_{j=1}^{N} w_j$. This DWR strategy directs computational resources toward challenging tasks where performance lags. Furthermore, the smoothing term ϵ guarantees that all tasks maintain a non-zero sampling probability, effectively preventing catastrophic forgetting of previously mastered skills while the agent tackles new challenges.

3.5 Asynchronous Parallel Data Prefetching

To address the I/O bottlenecks inherent in experience replay sampling and trajectory processing, we implement a data prefetching architecture based on asynchronous parallel logic. By maintaining a background asynchronous loop within the trainer that operates independently of the primary training task, AcceRL achieves a effective decoupling of data preparation from gradient updates. Functioning as a producer, this background process continuously monitors the distributed replay buffer and triggers cross-node trajectory sampling once the required threshold is met. This concurrency ensures that while the GPU executes gradient computations for the current batch, computational resources are simultaneously preparing the data for the subsequent iteration.

Furthermore, AcceRL simplifies the training critical path by delegating real-time advantage recomputation, observation tensorization, and batching operations to the background prefetcher. By stripping these complex inference and transformation tasks from the main loop, the system establishes a robust producer-consumer model where the trainer extracts ready-to-use super-batches directly from a local cache. This mechanism effectively achieves near-zero-latency data loading, maximizing hardware utilization and accelerating convergence in complex embodied tasks.

4 AcceRL with World Model

AcceRL can further integrate a world model, which consists of an observation model M_{obs} and a reward model M_{reward}, as shown in Figure 3b. Specifically, M_{obs} is responsible for predicting the next-frame observation (which corresponds to high-fidelity pixel-level images in our experiments) to simulate environment dynamics. To support this architecture, we deploy independent TrainerWorkers and InferenceWorkers for both M_{obs} and M_{reward}. Furthermore, the replay buffer is partitioned into two distinct components: a world model buffer $\mathcal{B}_{wm}$ to store real state transitions to train the world model, and an imaginary replay buffer $\mathcal{B}_{img}$ to store synthetic state trajectories generated by the world model for policy optimization.

The RolloutWorkers operate via an alternating sampling strategy where the generation of numerous imagined trajectory episodes within the world model is interleaved with the collection of a single trajectory from the real-world environment. The rollout in real environment serves as a collection process for both world model training data and initial state of imaginary. Initially, the worker dynamically selects an environment based on the failure rates. It then dispatches current observation o_t to the M_{policy} in the InferenceWorker Pool to get action a_t, which is executed to collect real sequences of observation o_{t+1} and reward r_t. Upon task completion, the worker stores the trajectory τ in episode buffer to serve as starting observation for imagination rollouts. Concurrently, the trajectory τ is pushed to the WMReplayBufferActor to facilitate the training of the world model's observation model M_{obs} and reward model M_{reward}.

To clearly distinguish simulated experiences from real-world data, we denote all variables generated during the world model rollout with a hat ($\hat{\cdot}$). During each world model rollout, the RolloutWorker generates a synthetic trajectory

episode by issuing inference requests to the InferenceWorker Pool. The process begins with an initial real frame o_t sampled from the local episode buffer, which serves as the starting observation for the imagination phase (i.e., $\hat{o}_t = o_t$). At each subsequent imaginary timestep $\tau_{img} \in [t, t+H]$, the worker first invokes the policy model M_{policy} within the pool to determine the imagined action $\hat{a}_t$. Subsequently, it requests the observation model M_{obs} to generate the predicted next frame $\hat{o}_{t+1}$. To mitigate the accumulation of errors inherent in long-term autoregressive world model predictions, these rollouts are restricted to a fixed horizon H, producing short-term trajectory episodes rather than full-length sequences. Concurrently, the predicted frames are transmitted to the reward model M_{reward} within the pool to estimate the success probability and the corresponding termination signal $\hat{done}$ for the current state. The collected imaginary trajectory episode $\hat{\tau}$ is then added to the imaginary replay buffer $\mathcal{B}_{img}$:

$$\hat{\tau} = \left(\hat{o}_{t:t+H+1}, \hat{a}_{t:t+H}, \hat{r}_{t:t+H}, \hat{\mu}_{t:t+H}, \hat{v}_{t:t+H}, \tilde{v}_{t+H+1}, \hat{done}\right). \tag{12}$$

Specifically, the reward for each imagined step is formulated as follows:

$$\hat{r}_\tau = M_{reward}(\hat{o}_{\tau+1}) - M_{reward}(\hat{o}_\tau), \tag{13}$$

which constitutes a potential-based reward structure. By defining the reward as the difference in success probability between successive states, this formulation ensures policy invariance. The primary advantage of this potential-based signal is its ability to accelerate the training process by providing dense, informative guidance without altering the agent's optimal policy, thereby maintaining the fundamental objective of the original task.

During the training phase, the TrainerWorker maintains three decoupled optimization processes with distinct update frequencies. For policy training, the trainer samples a synthetic data batch $\hat{\tau}$ from the imaginary replay buffer $\mathcal{B}_{img}$ to update the policy model M_{policy}. Meanwhile, the observation model M_{obs} is fine-tuned every T_{obs} cycles using the data sampled from the world model buffer $\mathcal{B}_{wm}$. For the reward model M_{reward}, the trainer performs regression on pairs (o_t, r_t) sampled from $\mathcal{B}_{wm}$ every T_{reward} steps to optimize M_{reward}. To minimize communication overhead, weights are transmitted to the corresponding inference nodes only when an actual update occurs. Furthermore, the heterogeneous inference workers utilize a dynamic batching protocol to handle concurrent requests for observation, reward, and policy models, which ensures the generation of high-fidelity dream trajectories without blocking the deployment pipeline.

5 Implementation Refinements for VLA RL

5.1 Vocabulary Slimming

To efficiently adapt the OpenVLA-OFT [26] (based on the Llama-2-7B language backbone) for RL, we prune the language model head lm_head to output only the action vocabulary (e.g., 256 bins), decoupling the policy from the full language lexicon. This reduces VRAM and compute costs while preventing probability mass from leaking to non-action tokens.

AcceRL employs an in-place replacement strategy to crop the original, massive output layer lm_head into a compact linear layer containing only action-related tokens. Specifically, the system first locates the start index and end index of the action tokens in the original vocabulary, then clones the corresponding weights from the original $lm_head.weight$ via slicing. Finally, a new linear layer with an output dimension of $N_{actions}$ is created, and the extracted weights are migrated to this layer, directly overwriting the original language model head.

By slimming the vocabluary, the model only needs to output action commands and does not need to process language text, provides the following advantages:

- VRAM Efficiency: Reducing the weight matrix from $[V, d_{model}]$ (where $V \approx 32k$) to $[N_{actions}, d_{model}]$ significantly reduces the existing occupancy of gradients and optimizer states.
- Increased throughput: Eliminating most of the computational overhead of the last layer significantly speeds up the iteration frequency of each training step.
- Noise Reduction: Physically isolating non-action tokens prevents the model from assigning probability to irrelevant vocabulary, focusing the policy distribution and speeding up convergence.

5.2 Value Head Design

The Value Head is designed to estimate the state value $V(s_t)$ by focusing on decision-critical information within the VLA's hidden states. In AcceRL, we implement an Action-Aware Attention Pooling mechanism. Given a sequence of

all action-related hidden states $\mathbf{H} = \{\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_N\}$ extracted from the last hidden layer of the VLA, we compute an attention score e_i for each token:

$$e_i = \mathbf{w}^\top \mathbf{h}_i + b, \tag{14}$$

where $\mathbf{h}_i$ has been detached. The normalized attention weights α_i are then derived via a softmax operation, and the pooled representation $\mathbf{z}_{pool}$ is calculated as a weighted sum:

$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^{N} \exp(e_j)}, \qquad \mathbf{z}_{pool} = \sum_{i=1}^{N} \alpha_i \mathbf{h}_i \tag{15}$$

To explicitly account for temporal constraints, we incorporate a step embedding $\mathbf{e}_{step} \in \mathbb{R}^D$. This embedding maps the current episode step t to a high-dimensional vector, which is element-wise added to the pooled feature. The final state value is estimated through a MLP:

$$V(s_t) = \text{MLP}(\mathbf{z}_{pool} + \mathbf{e}_{step}) \tag{16}$$

5.3 Token-level Policy Optimization

Defining the granularity of the optimization objective is critical for training stability, leading us to distinguish between three primary approaches: chunk-level, action-level, and our adopted token-level optimization.

Standard chunk-level methods treat the entire sequence of K tokens as a single unified action. The joint probability is calculated as $P(A|s) = \prod_{k=1}^{K} \pi(a_k|s, a_{<k})$, which frequently leads to severe numerical instability. Because the joint probability is a product of multiple values between 0 and 1, the resulting magnitude can become infinitesimally small, causing the PPO importance sampling ratio $r_t(\theta) = \frac{\pi_\theta(A|s)}{\pi_{\theta_{old}}(A|s)}$ to fluctuate wildly. Such fluctuations often trigger the PPO clipping mechanism, leading to a total loss of gradient signals and stalled training.

Alternatively, an action-level approach might compute the loss based on the mean or aggregated probability of the tokens within a single action dimension. However, this still risks blurring the fine-grained causal dependencies between individual tokens in the VLA's auto-regressive generation process.

In contrast, we implement a token-level optimization strategy. By treating each token as an independent decision point, the importance sampling ratio is redefined for each individual token:

$$r_{t,k}(\theta) = \frac{\pi_\theta(a_k|s, a_{<k})}{\pi_{\theta_{old}}(a_k|s, a_{<k})} \tag{17}$$

By calculating the clip loss at the token level, we prevent outlier tokens from causing the entire action chunk's gradient to be clipped, thereby retaining a much higher density of training signals. This approach ensures numerical robustness by avoiding the vanishing product problem and provides precise credit assignment to specific components of the action sequence. Furthermore, the dependency $a_{<k}$ is inherently maintained by the VLA's causal self-attention mechanism, ensuring that each token-level ratio accurately reflects the conditional probability shift while enabling stable policy evolution on large scale VLA backbones.

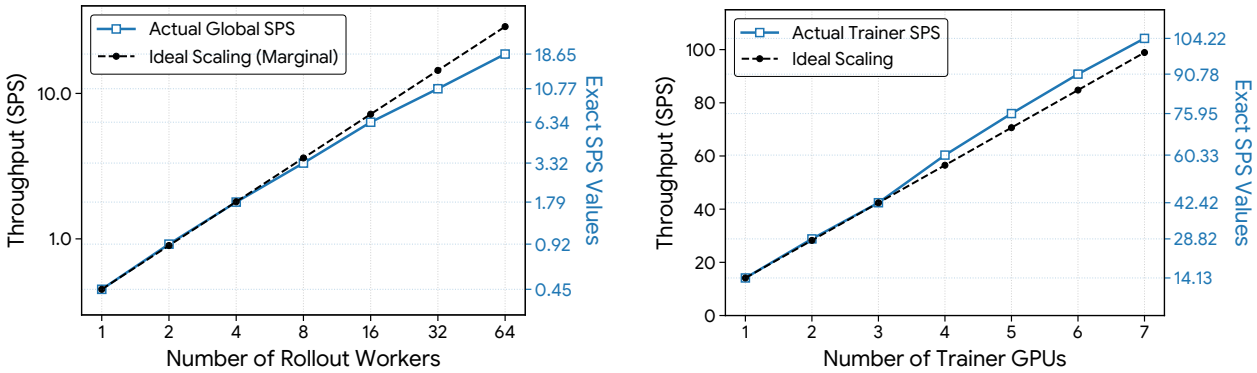
6 Experiments

6.1 Experiment Setup

Architecture The framework consists of three core components for world-model-based reinforcement learning. The policy model M_{policy} utilizes the OpenVLA-OFT [26] architecture to process observation images and proprioceptive states into discretized action tokens and employs action chunking for temporal consistency. The observation model M_{obs} is a diffusion-based transition model built upon the DIAMOND (DIffusion As a Model Of eNvironment Dreams) [23] architecture. Unlike traditional world models that compress environment dynamics into discrete latent variables, DIAMOND leverages a diffusion process to preserve fine-grained visual details. This enables M_{obs} to generate high-fidelity, synthesized next-frame images conditioned on historical observation sequences and current action chunks. Acting as a virtual referee, the reward model M_{reward} is a binary classifier modified from OpenVLA-OFT that evaluates these synthetic frames to estimate success probabilities and termination signals.

Experimental Benchmark We evaluate our system using the LIBERO [1] benchmark, which provides a diverse set of challenges across various domains. The experimental configuration includes multiple subsets such as LIBERO-Spatial, LIBERO-Object, LIBERO-Long, and LIBERO-Goal. All simulations are conducted within the MuJoCo physics engine, utilizing OSMesa for off-screen software rendering. This setup facilitates large-scale parallel sampling across high-performance computing clusters that lack dedicated display hardware.

6.2 Throughput Comparison



(a) SPS vs. Number of Rollout Workers (Log Scale)

(b) Trainer SPS vs. Number of Trainer GPUs

Figure 4: **Scalability performance of the AcceRL framework with decoupled data labels. (a)** Throughput scaling demonstrates near-linear performance up to 64 rollout workers. **(b)** Trainer scalability shows that the actual trainer SPS closely tracks the ideal marginal scaling curve when scaling up to 7 GPUs. Both configurations effectively maintain high hardware utilization without suffering from severe communication overhead.

Table 1: **Scaling performance and hardware utilization of AcceRL.** The data illustrates a super-linear increase in training throughput (SPS) while maintaining consistently high GPU utilization ($> 94\%$) at different scale points.

GPUs	Throughput (SPS)	GPU Util. (%)
1	14.13	96.45
2	28.82	97.17
3	42.42	94.22
4	60.33	98.36
5	75.95	96.72
6	90.78	96.63
7	104.22	95.07

We first evaluate the training throughput and scalability of AcceRL. By decoupling environment interaction from action generation and using centralized dynamic batching, AcceRL effectively reduces GPU "bubbles" caused by long-tail inference latency. To demonstrate the efficiency of this design, Figure 4 shows the scaling performance of the system. Notably, when scaling the trainer to 7 GPUs(H200), AcceRL exhibits a super-linear scaling effect, reaching 104.22 SPS (samples per second). This exceptional throughput is primarily driven by our distributed update strategy utilizing DeepSpeed ZeRO-2. By partitioning optimizer states and gradients across multiple GPUs, ZeRO-2 significantly reduces the memory footprint per device. As the cluster expands, this massive memory efficiency allows the system to accommodate a larger micro-batch size per GPU without out-of-memory errors. The increased batch size heavily amortizes kernel launch overheads and maximizes Tensor Core utilization (arithmetic intensity), allowing the actual throughput to consistently surpass the ideal marginal scaling curve while maintaining a high GPU utilization of over 94%. This confirms that our framework successfully translates memory optimizations into sheer computational acceleration, completely avoiding heavy communication bottlenecks.

6.3 Task Performance on LIBERO

To evaluate the effectiveness of the proposed AcceRL framework, we compare its task success rates against the standard OpenVLA-OFT baseline, as well as recent state-of-the-art RL-based frameworks including RLinf-VLA [5] and SimpleVLA-RL [15], across the four diverse LIBERO task suites. OpenVLA-OFT uses Orthogonal Fine-Tuning to adapt Vision-Language-Action (VLA) models, serving as a highly competitive supervised learning reference for our evaluation.

As summarized in Table 2, AcceRL consistently outperforms the baseline methods across the task categories. While OpenVLA-OFT performs well on shorter and simpler tasks, its success rate drops to 90.7% on the LIBERO-Long tasks. This performance drop occurs because supervised imitation learning is highly sensitive to error accumulation over long

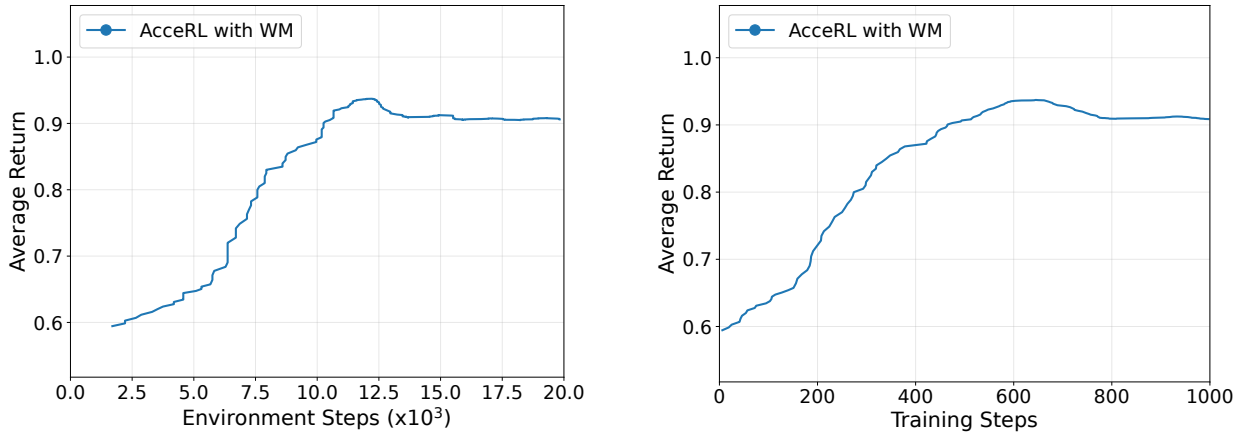
Table 2: **Task performance comparison on the LIBERO benchmark.** Our proposed AcceRL framework achieves superior success rates across almost all task categories. Notably, AcceRL achieves a 99.1% success rate in the Long category, demonstrating that our reinforcement learning approach effectively maintains stability in long-horizon tasks where traditional supervised fine-tuning often struggles.

Method	Spatial (%)	Object (%)	Goal (%)	Long (%)
Ours (AcceRL)	**99.6**	**100.0**	98.8	**99.1**
SimpleVLA-RL [15]	99.4	99.8	**99.2**	98.5
RLinf-VLA [5]	99.4	99.8	98.8	94.0
OpenVLA-OFT	96.2	98.3	96.2	90.7

sequences; a minor mistake in early steps can easily lead to task failure. Although recent RL-augmented frameworks like RLinf and SimpleVLA-RL significantly improve upon this supervised baseline, they still exhibit performance degradation in the most demanding long-horizon scenarios.

In contrast, our AcceRL framework achieves a striking 99.1% success rate on the same long-horizon tasks. By formulating the manipulation problem through a distributed reinforcement learning paradigm, AcceRL learns to maximize long-term rewards rather than just mimicking single-step actions. This mechanism allows the policy to recover from minor deviations and maintain stable, continuous execution throughout complex, multi-stage tasks, demonstrating superior robustness even when compared to concurrent RL fine-tuning systems.

6.4 Performance Evaluation of AcceRL-WM on the LIBERO



(a) Average Return vs. Environment Steps

(b) Average Return vs. Training Steps

Figure 5: **Learning curves of the proposed AcceRL framework with World Model (WM) integration. (a)** The evaluation average return plotted against environment steps demonstrates exceptional sample efficiency. To clearly illustrate the effectiveness of RL fine-tuning, the policy is deliberately initialized from a suboptimal checkpoint trained with limited demonstrations (starting at $\sim$0.6 return). By leveraging imagination, the policy rapidly breaks through the 0.8 return threshold within merely 10,000 real-world interactions. **(b)** The evaluation performance against training steps validates the rapid convergence speed. Driven by analytic gradients from the differentiable WM, the policy reaches a stable, near-optimal plateau in fewer than 400 updates.

For the RL fine-tuning phase, we intentionally initialize the policy with a suboptimal checkpoint (pre-trained via Behavior Cloning on a limited subset of demonstrations) to provide sufficient performance headroom for evaluating the learning efficiency of the proposed framework. As illustrated in Figure 5, the integration of a differentiable World Model (WM) significantly accelerates the fine-tuning process of the VLA policy.

First, our framework demonstrates exceptional sample efficiency. As shown in Figure 5a, the policy rapidly breaks through the 0.8 average return threshold within only $10,000$ environment steps. This efficiency is primarily attributed to the WM's "imagination" capability. Because our framework completely decouples the world model from the policy, the agent is not restricted to learning in a compressed latent space. Instead, it performs extensive pixel-level rollouts, interacting directly with high-fidelity synthetic image sequences generated by the diffusion model.

Second, Figure 5b validates the rapid convergence speed. Unlike traditional RL training that heavily struggles with sparse and delayed environmental feedback, AcceRL-WM leverages dense, potential-based reward signals computed by the reward model for every imagined transition. Combined with the massive volume of high-fidelity synthetic trajectories from the diffusion model, this provides a highly informative and dense optimization landscape. Consequently, the policy is guided via efficient policy gradients to reach a stable, near-optimal performance plateau in fewer than 400 training updates.

6.5 Ablation Study of AcceRL without Value Recomputation

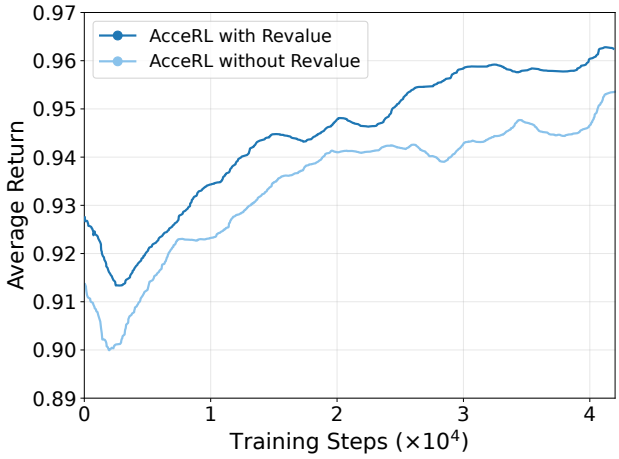


Figure 6: **Ablation study of the AcceRL framework regarding the value recomputation mechanism.** The comparison highlights the critical role of recomputing values in maintaining training stability. AcceRL with Revalue (dark blue) effectively handles data staleness in low-rollout scenarios, while AcceRL without Revalue (light blue) exhibits significant performance drops and higher variance due to misaligned value targets.

While the previous results demonstrate the overall efficiency of the AcceRL framework, its performance in asynchronous distributed settings relies heavily on maintaining consistency between the actor and the critic. A significant technical challenge in such architectures is data staleness: since rollout and training occur in parallel, the experiences stored in the replay buffer were often evaluated by an earlier, outdated version of the critic network. To mitigate the resulting gradient noise and training instability, we introduced a value recomputation mechanism that updates value targets using the most recent critic before each gradient update. As illustrated in Figure 6, the value recomputation mechanism serves as a critical stabilizer for off-policy VLA fine-tuning. The full AcceRL framework (dark blue) effectively handles data staleness, maintaining high average returns with minimal variance even in resource-constrained settings.

In contrast, the version without recomputation (light blue) exhibits performance oscillations and a lower convergence ceiling. This instability arises from misaligned value targets, where the policy is updated based on evaluations that no longer reflect the current state of the critic. By ensuring that the policy is always guided by the most accurate and up-to-date value estimates, the recomputation mechanism prevents error accumulation and ensures robust policy improvement throughout the training process.

6.6 Ablation Study of Gaussian Importance sampling Policy Optimization

Beyond system-level optimizations, the choice of policy optimization objective is critical for stability in asynchronous settings. As analyzed in Section 3.3, the inherent policy lag in decoupled architectures can destabilize standard PPO and make it perform poorly. To mitigate this, we employ our previously proposed method GIPO [25], which replaces hard clipping with a Gaussian trust weight to softly damp extreme importance ratios.

As illustrated in Figure 7, we plot the raw evaluation returns to explicitly visualize the training dynamics. A prominent, rapid surge is observed at the very beginning of the training (near 0k steps). This phenomenon is a statistical artifact inherent to the Episode-Level Long-Tail in our asynchronous rollout mechanism. During the initial phase, successful trajectories typically have shorter horizons and are therefore completed and reported to the buffer faster than unsuccessful ones that persist through the full task duration [7]. This leads to a temporary, optimistic skew in success rates before the system reaches a representative sample distribution.

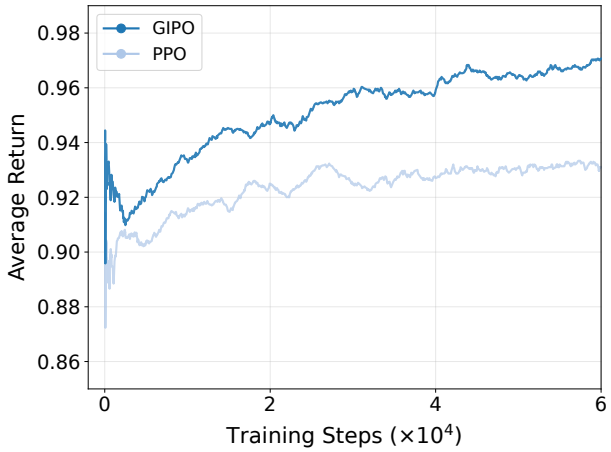


Figure 7: **Ablation study of the AcceRL framework regarding the GIPO algorithm.** The curves highlight the stability of GIPO (dark blue) compared to standard PPO (light blue). GIPO maintains a robust, high-performance trajectory, whereas PPO exhibited oscillatory behavior and inferior performance due to policy lag.

Once stabilized, the comparison reveals a clear divergence. The AcceRL framework equipped with GIPO (dark blue) maintains a tightly clustered, high-performance trajectory. In contrast, the AcceRL variant utilizing standard PPO (light blue) exhibits severe performance oscillations and significantly lower sample efficiency. This instability is primarily due to PPO's reliance on a hard clipping mechanism, which becomes counterproductive in asynchronous settings where data generation can be delayed or experiences may become stale. In such scenarios, a large proportion of the sampled data triggers the clipping threshold, resulting in ineffective stale updates with zero gradients that fail to improve the policy.

To overcome these algorithmic limitations, GIPO replaces hard clipping with a smooth log-ratio trust-weighted surrogate. This design inherently provides a symmetric and continuous penalty, implicitly enforcing a tunable bound on the policy update magnitude without abruptly zeroing out useful gradients. Furthermore, by achieving a superior bias-variance trade-off, GIPO provides formal guarantees of robustness and stability under finite-sample estimation, even when learning from highly stale replay data. By effectively mitigating this data utilization collapse, GIPO demonstrates a dramatic acceleration in convergence: it achieves a performance level at approximately 8,000 steps that standard PPO requires 60,000 steps to reach, representing a roughly 7.5-fold improvement in sample efficiency.

7 Conclusion

This paper presents AcceRL, an asynchronous reinforcement learning framework optimized for large scale VLA. The core strength of the framework lies in its trainable and plug and play world model module. This integration facilitates high fidelity imagination rollouts that allow the system to transcend the frequency constraints of physical simulators while significantly enhancing sample efficiency. Furthermore AcceRL achieves superior performance by leveraging a fully decoupled architecture for rollout and inference and training. This design effectively eliminates systemic bottlenecks such as synchronization overhead and memory constraints. By ensuring high hardware utilization, it unlocks near-linear scalability for distributed rollouts and super-linear throughput for the trainer as the cluster expands. Although the framework demonstrates high efficiency it does not yet support the post training of large scale language models. Future research will focus on extending the architecture to accommodate full scale model alignment.

References

[1] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.

[2] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: international conference for high performance computing, networking, storage and analysis*, pages 1–16. IEEE, 2020.

[3] Guangming Sheng, Chi Zhang, Ziqian Ye, Xudong Wu, Wei Zhang, Ruobing Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

[4] Chao Yu, Y. Wang, Z. Guo, H. Lin, S. Xu, H. Zang, Q. Zhang, Y. Wu, C. Zhu, J. Hu, et al. RLinf: Flexible and efficient large-scale reinforcement learning via macro-to-micro flow transformation. *arXiv preprint arXiv:2509.15965*, 2025.

[5] Hongzhi Zang, Mingjie Wei, Si Xu, Yongji Wu, Zhen Guo, Yuanqing Wang, Hao Lin, Peihong Wang, Liangzhi Shi, Yuqing Xie, Zhexuan Xu, Zhihao Liu, Kang Chen, Wenhao Tang, Quanlu Zhang, Weinan Zhang, Chao Yu, and Yu Wang. Rlinf-vla: A unified and efficient framework for reinforcement learning of vision-language-action models, 2026.

[6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[7] James Cipar, Qirong Ho, Jin Kyu Kim, Seunghak Lee, Gregory R. Ganger, Garth Gibson, Kimberly Keeton, and Eric Xing. Solving the straggler problem with bounded staleness. In *14th Workshop on Hot Topics in Operating Systems (HotOS XIV)*, Santa Ana Pueblo, NM, May 2013. USENIX Association.

[8] Eric Liang, Richard Liaw, Philipp Moritz, Robert Nishihara, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[9] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architecture. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, pages 1407–1416. PMLR, 2018.

[10] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. Laminar: A scalable asynchronous rl post-training framework. *arXiv preprint arXiv:2510.12633*, 2025.

[11] Zhong Guan, Haoran Sun, Yongjian Guo, Shuai Di, Xiaodong Bai, Jing Long, Tianyun Zhao, Mingxi Luo, Chen Zhou, Yucheng Guo, Qiming Yang, Wanting Xu, Wen Huang, Yunxuan Ma, Hongke Zhao, Likang Wu, Xiaotie Deng, Xi Xiao, Sheng Wen, Yicheng Gong, and Junwu Xiong. Rl-vla^3: Reinforcement learning vla accelerating via full asynchronism. *arXiv preprint arXiv:2602.05765*, 2026.

[12] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-1: Robotics transformer for real-world control at scale. In *arXiv preprint arXiv:2212.06817*, 2023.

[13] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.

[14] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[15] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. Simplevla-rl: Scaling vla training via reinforcement learning, 2025.

[16] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[17] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[18] Yixian Zhang, Shu'ang Yu, Tonghe Zhang, Mo Guang, Haojia Hui, Kaiwen Long, Yu Wang, Chao Yu, and Wenbo Ding. Sac flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling. *arXiv preprint arXiv:2509.25756v3*, 2026.

[19] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[20] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2024.

[21] Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments. In *International Conference on Learning Representations (ICLR)*, 2024.

[22] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.

[23] Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. *Advances in Neural Information Processing Systems*, 37:58757–58791, 2024.

[24] Zhennan Jiang, Shangqing Zhou, Yuhui Chen, Tianxing Zhou, Haoran Li, Yu Wang, Yutong Jiang, Zefang Huang, Mingjie Wei, Zhen Guo, Hao Lin, Quanlu Zhang, Chao Yu, and Dongbin Zhao. Wovr: World models as reliable simulators for post-training vla policies with rl. *arXiv preprint arXiv:2602.13977v1*, 2026.

[25] Chengxuan Lu, Zhenquan Zhang, Shukuan Wang, Qunzhi Lin, Baigui Sun, and Yang Liu. Gipo: Gaussian importance sampling policy optimization. *arXiv preprint arXiv:2603.03955*, 2026.

[26] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success, 2025.